Programming Assignment 3: Mini Packet Analyzer

Weight: 5 marks

Duration: 2 Weeks

Programming Language: Python

Design and implement a simplified packet-analysis tool similar to Wireshark. The tool should capture network packets and decode their contents. The core packet-header parsing must be implemented by you rather than delegated entirely to a packet-decoding library.

Allowed Libraries: Students may use Python standard libraries such as socket, struct, binascii, ipaddress, time, datetime, and other general-purpose libraries. A library may be used to obtain the raw packet bytes, but the **IPv4, TCP, and UDP header parsing must be implemented by the student**. Libraries that automatically decode protocol headers, such as Scapy, PyShark, or equivalent packet-parsing libraries, may not be used for the core packet-analysis functionality.

1. Packet Capture

Your tool must:

- Capture packets transmitted or received by the host.
- Number and timestamp captured packets.
- Display the packet size.
- Allow an individual packet to be selected for detailed inspection.

Packet capture must only be performed on systems and networks you own or are authorized to analyze.

2. IP, TCP, and UDP Header Analysis

For each IPv4 packet, extract relevant header fields.

IP

- Version and header length
- Total length and TTL
- Protocol and checksum
- Source and destination IP addresses

TCP

- Source and destination ports

- Sequence and acknowledgment numbers
- Header length and flags
- Window size and checksum
- Payload length

UDP

- Source and destination ports
- Length and checksum
- Payload length

The program must automatically identify whether an IP packet contains TCP or UDP.

3. Packet Filtering and Display

Provide a summary of captured packets containing:

- Packet number
- Source and destination IP addresses
- Protocol
- Source and destination ports
- Packet length

Support filtering by:

- TCP or UDP
- Source/destination IP
- Source/destination port

Selecting a packet should show its detailed **IP header, TCP/UDP header, and payload**.

Important Requirement

The educational objective is to understand **how a packet is represented as bytes**. Therefore, students must implement the core **IPv4, TCP, and UDP header parsing themselves** and be able to explain how fields are extracted from the raw packet.

A command-line interface is sufficient. A graphical interface is not required.

Sample Packet Output

The following illustrates the expected level of detail. **Students do not need to reproduce this exact format.**

```
Packet #17
Time: 14:32:08.421
Packet Length: 92 bytes

IP Header
--------------------------------
Version:              4
Header Length:        20 bytes
Total Length:         92 bytes
 TTL:                  64
 Protocol:             TCP
 Header Checksum:      0x8A31
 Source IP:            192.168.1.10
 Destination IP:       192.168.1.20

 TCP Header
 --------------------------------
 Source Port:          49152
 Destination Port:     8080
 Sequence Number:      105482
 Acknowledgment:       208713
 Header Length:        20 bytes
 Flags:                PSH, ACK
 Window Size:          65535
 Checksum:             0x2F81
 Payload Length:       52 bytes

 Payload
 --------------------------------
 Hex:
 48 45 4C 4C 4F 20 53 45 52 56 45 52 ...

 Text:
 HELLO SERVER ...
```

A packet without application data should show an empty payload or a payload length of zero rather than attempting to interpret data that is not present.